

Self-Evolving AI: A Unified Framework for Generation, Evaluation, Memory, and Improvement

A Survey of Self-Improvement Loops in Large Language Models,
Evolutionary Computation, and Autonomous Agents

Self Evolve Research Team

<https://github.com/copizzah/awesome-evolution>

May 2026

Abstract

AI self-evolution is not a single technique but a convergence of five fundamental loops: specification-to-execution, search, evaluator, reflection, and population. This survey systematically reviews the emerging landscape of self-improving AI systems, spanning 19+ open-source projects and 17+ foundational papers across five research communities—AutoML, Neural Architecture Search (NAS), LLM self-improvement, evolutionary computation, and autonomous agent frameworks. We propose a unified taxonomy that classifies existing work along the five evolution loops, revealing that progress in each community independently rediscovers mechanisms that others have formalized. We analyze self-improvement methods at both inference time (Self-Refine, Reflexion, Self-Debug) and training time (SPIN, STaR, Constitutional AI), trace the emergence of LLM-driven evolutionary optimization from OPRO to AlphaEvolve, and map the frontier of agent self-evolution from fixed architectures to open-ended self-modifying systems (Darwin Gödel Machine, Gödel Agent, Absolute Zero). We identify critical gaps in current agent frameworks—the absence of persistent evaluation, lineage tracking, and cross-run memory—and position Self-Evolve as the missing evolution layer above existing orchestration frameworks.

Keywords: Self-Evolution, LLM Self-Improvement, Evolutionary Computation, Agent Evolution, AutoML, Neural Architecture Search, Autonomous Agents

Contents

1 Introduction

The history of artificial intelligence is characterized by alternating waves of static, hand-designed systems and adaptive, self-improving ones. Early expert systems required manual knowledge engineering; statistical machine learning introduced automated parameter fitting; deep learning automated feature engineering; and now, large language models (LLMs) are beginning to automate their own improvement. We stand at the threshold of a new paradigm: **self-evolving AI**—systems that can generate, evaluate, remember, and improve upon their own behavior without explicit human intervention at each step.

1.1 From Static AI to Self-Evolving AI

The trajectory from static to self-evolving AI can be understood through a series of expanding automation scopes:

1. **Static systems** (pre-2012): Hand-crafted rules, features, and architectures. Every component designed by humans.
2. **Learned representations** (2012–2017): Deep learning automated feature engineering, but architectures and training procedures remained hand-designed.
3. **Architecture search** (2017–2020): Neural Architecture Search (NAS) and AutoML began automating the design of model architectures and ML pipelines.
4. **Prompt-driven systems** (2020–2023): LLMs enabled natural language specification of tasks, with systems like AutoML-GPT converting task descriptions into complete ML pipelines.
5. **Self-evolving systems** (2023–present): Systems that can modify their own code, prompts, workflows, and architectures through iterative cycles of generation, evaluation, and improvement.

Each transition automated what was previously considered an inherently human creative act. Self-evolving AI represents the current frontier: systems that improve their own improvement process.

1.2 What is AI Self-Evolution?

We define **AI self-evolution** as the capacity of an AI system to autonomously improve its performance through iterative cycles that involve:

- **Generation:** Producing candidate solutions, architectures, prompts, code, or agent behaviors.
- **Evaluation:** Scoring candidates against task objectives, benchmarks, or deployed metrics.
- **Memory:** Storing successful and failed attempts for future reference.
- **Improvement:** Using accumulated experience to produce better candidates in subsequent iterations.

This definition deliberately spans multiple research communities. A prompt optimization system (OPRO) and an agent architecture search system (ADAS) both exhibit self-evolution, even though they operate on different substrates (text prompts vs. agent code) and use different mechanisms (LLM-as-optimizer vs. foundation model-guided search).

1.3 Relation to Existing Concepts

AI self-evolution intersects with but is distinct from several established concepts:

- **Continual learning** focuses on learning from sequential data streams without catastrophic forgetting. Self-evolution focuses on improving the *system itself* (its prompts, code, architecture), not just updating a model on new data.

- **Online learning** updates model parameters from streaming data. Self-evolution may involve parameter updates but also encompasses non-parametric improvements (prompt refinement, code mutation, workflow restructuring).
- **Self-supervised learning** creates training signals from the data itself. Self-evolution creates improvement signals from the system’s own execution trace and evaluation feedback.
- **Meta-learning** learns how to learn. Self-evolution is a form of meta-learning but extends beyond parameter adaptation to include structural and behavioral modification of the agent itself.

1.4 Contributions

This survey makes the following contributions:

1. A **unified taxonomy of five evolution loops** that classifies self-evolution mechanisms across five research communities (Section ??).
2. A systematic analysis of **19+ open-source projects** with detailed technical comparisons (Sections ??–??).
3. A **cross-domain perspective** that unifies AutoML, NAS, LLM self-improvement, evolutionary computation, and agent frameworks under a single framework.
4. An analysis of the **researcher ecosystem** including institutional relationships and citation networks (Section ??).
5. Identification of **open problems** and a roadmap for building production self-evolving systems (Section ??).

1.5 Paper Structure

The remainder of this paper is organized as follows. Section ?? presents our five-loop taxonomy. Section ?? reviews self-improvement methods at inference and training time. Section ?? covers evolutionary code optimization and algorithm discovery. Section ?? surveys agent self-evolution systems. Section ?? reviews agent frameworks and their evolution capability gaps. Section ?? examines the intersection of AutoML, NAS, and LLMs. Section ?? maps the research community. Section ?? discusses open problems and future directions. Section ?? concludes.

2 Taxonomy: Five Evolution Loops

We propose that self-evolving AI systems can be understood through the lens of five fundamental evolution loops. Each loop represents a distinct pattern of self-improvement that has been independently discovered and refined by different research communities. Crucially, these loops are not mutually exclusive—the most powerful self-evolving systems combine multiple loops simultaneously.

2.1 Specification-to-Execution Loop

The specification-to-execution loop translates natural language goals into runnable machine learning pipelines or agent workflows. This loop operates at the highest level of abstraction, bridging the gap between human intent and executable systems.

Representative systems: AutoML-GPT (?) uses GPT as a natural language bridge, converting task cards, model cards, and data cards into data processing, model architecture, hyperparameter optimization, and training log prediction. AutoML-Agent (?) decomposes the full AutoML pipeline into specialized agents for data retrieval, preprocessing, modeling, evaluation, and deployment, combining retrieval-augmented planning with multi-stage validation.

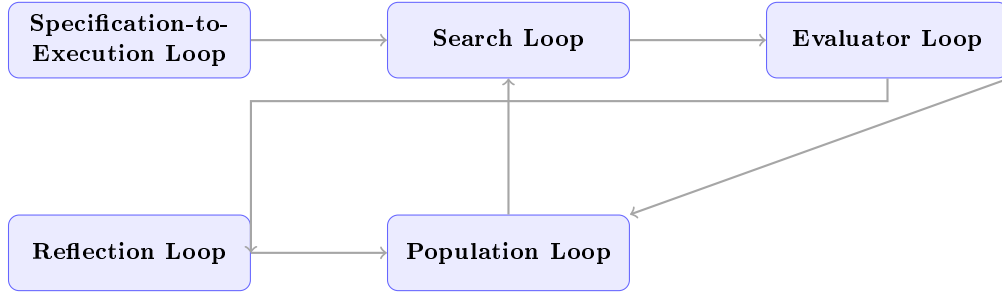


Figure 1: The five evolution loops and their interconnections. The specification-to-execution loop translates goals into runnable pipelines; the search loop explores candidate spaces; the evaluator loop provides fitness signals; the reflection loop converts failures into memory; and the population loop maintains diverse candidates for selection and recombination.

Loop dynamics: The user specifies a task in natural language → the system plans a pipeline → executes the pipeline → evaluates the result → refines its understanding of the specification. Each iteration improves both the output quality and the system’s ability to interpret similar specifications in the future.

Key insight: This loop extends AutoML from fixed toolboxes to prompt-driven planning and execution pipelines. The main gap in current implementations is **persistent memory**—most systems start from scratch for each new task rather than accumulating experience across specifications.

2.2 Search Loop

The search loop explores the space of possible solutions—architectures, prompts, code, agent configurations, or hyperparameters—using LLMs as intelligent search operators.

Representative systems: OPRO (?) uses LLMs as general-purpose optimizers, where the history of solutions and their scores serves as the optimization landscape. The LLM reads which candidates performed well and which did not, then generates new candidates that are likely to score higher. EvoPrompting (?) treats LLMs as evolutionary operators for neural architecture code. ADAS (?) automates the search over the design space of agentic systems. OpenEvolve (?) provides an open-source implementation of evolutionary code optimization.

Search spaces: The search loop can operate over discrete spaces (prompts, code, architectural choices) or continuous spaces (hyperparameters, embeddings). What distinguishes LLM-driven search from traditional optimization is the model’s ability to understand the *structure* of the search space and propose semantically meaningful variations rather than random perturbations.

Key mechanisms:

- **Candidate generation:** LLM proposes new candidates based on historical performance.
- **Temperature-controlled exploration:** Higher temperature increases diversity; lower temperature exploits known-good patterns.
- **Few-shot guidance:** Historical solutions serve as in-context examples that steer generation.
- **Deduplication:** Hash-based filtering prevents revisiting previously explored candidates.

2.3 Evaluator Loop

The evaluator loop provides the fitness signal that drives all other loops. Without reliable evaluation, generation is blind and search is random.

Representative systems: FunSearch (?) combines LLMs with automated evaluators to evolve mathematical programs, achieving the first improvement over Strassen’s matrix multiplication algorithm in 56

years. Self-Debug (?) uses execution feedback as a first-class self-repair signal: the program runs, fails, and the error trace becomes the input for the next iteration of improvement. Science-CodeEvolve uses task-specific scientific metrics to evaluate evolved code.

Evaluation modalities:

- **Automated testing:** Unit tests, integration tests, and property-based tests provide binary or scalar scores.
- **Benchmark suites:** Standardized datasets (HumanEval, GSM8K, SWE-bench) enable cross-system comparison.
- **Execution traces:** Runtime behavior, error messages, and performance profiles provide rich diagnostic feedback.
- **Deployment metrics:** Production indicators (latency, cost, user satisfaction) ground evaluation in real-world performance.

Key insight: The evaluator loop is the bottleneck of self-evolution. As systems improve on standard benchmarks, the marginal information gain from each evaluation decreases, creating pressure for more sophisticated evaluation strategies.

2.4 Reflection Loop

The reflection loop converts failures into memory, feedback, and revised candidate generation strategies. Unlike the evaluator loop (which produces scores), the reflection loop produces *explanations* of why a candidate failed and *suggestions* for how to improve it.

Representative systems: Reflexion (?) introduces verbal reinforcement learning, where agents learn from linguistic feedback stored in memory without any weight updates. The system maintains three specialized models: an Actor that generates actions, an Evaluator that scores them, and a Self-Reflection module that generates verbal feedback explaining what went wrong. This achieves 91% pass@1 on HumanEval, surpassing GPT-4’s 80%.

Self-Refine (?) demonstrates that the same LLM can generate an output, critique it, and iteratively rewrite it—all at inference time without any training. STaR (?) extends this to training time by using self-generated reasoning traces as training data.

Reflection mechanisms:

- **Verbal feedback:** Natural language descriptions of what went wrong and what to do differently.
- **Episodic memory:** Growing buffers of past attempts and their outcomes.
- **Error categorization:** Classifying failures by type (syntax errors, logical errors, runtime errors) enables targeted repair strategies.

Key insight: Reflection operates at inference time without weight updates, making it the most accessible form of self-improvement. However, it is limited by the quality of the reflection signal—a model cannot reflect on aspects of its behavior that it cannot articulate.

2.5 Population Loop

The population loop maintains multiple candidate solutions simultaneously, enabling selection, mutation, recombination, and specialization across a diverse set of alternatives. This is the most computationally expensive but potentially most powerful loop.

Representative systems: AlphaEvolve (?) uses MAP-Elites quality-diversity search with a dual-model architecture (Gemini Flash for breadth exploration, Gemini Pro for depth). It maintains an archive of diverse, high-quality solutions and achieved the first improvement over Strassen’s algorithm in 56 years (48 multiplications for 4×4 complex matrix multiplication). At Google, it recovered 0.7% of worldwide compute through improved data center scheduling.

LLaMEA (?) evolves not task solutions but *optimizers themselves*—the most direct instantiation of “improving the improver.” AgentEvolver applies population-based evolution to agent workflow optimization. The Darwin Gödel Machine (?) maintains an open-ended archive of agent implementations, enabling cumulative improvement without local optima traps (SWE-bench: 20% → 50%, Polyglot: 14.2% → 30.7%).

Population management strategies:

- **MAP-Elites:** Maintains the best solution per behavioral niche, mapping the solution landscape.
- **Quality-diversity:** Balances performance against behavioral diversity.
- **Open-ended archives:** Growing collections that never converge to a single optimum.
- **Fitness-proportionate selection:** Sampling probability proportional to performance.

Key insight: Population-based approaches are uniquely suited for open-ended exploration because they maintain diverse alternatives rather than converging to a single optimum. The cost is computational—each candidate must be independently evaluated—but the benefit is robustness and the potential for serendipitous discovery.

2.6 Cross-Loop Comparison

Table ?? summarizes the key characteristics of each evolution loop.

Table 1: Comparison of the five evolution loops.

Loop	What Evolves	Generator	Evaluator	Memory
Spec-to-Exec	ML pipelines, agent workflows	Planner + task agents	Dataset metrics, deployment checks	Pipeline history
Search	Prompts, code, architectures	LLM as search operator	Task-specific scores	Candidate archive
Evaluator	Evaluation strategies	Test generators	Meta-evaluation	Failure patterns
Reflection	Reasoning strategies	Self-critique LLM	Verbal feedback	Episodic memory buffer
Population	Agent implementations, algorithms	LLM mutation/crossover	Fitness functions	Population archive

3 Self-Improvement Methods

Self-improvement methods form the core of AI self-evolution, enabling systems to enhance their performance through iterative feedback. We categorize these methods along two axes: whether they operate at inference time (without modifying model weights) or training time (with weight updates), and whether feedback comes from external sources or the model itself.

3.1 Inference-Time Self-Improvement

Inference-time methods improve outputs without modifying the underlying model. They are the most accessible form of self-improvement because they require no training infrastructure.

3.1.1 Self-Refine

Self-Refine (?) demonstrates that a single LLM can serve as both generator and critic. The process operates in iterative rounds:

1. **Generate:** The LLM produces an initial output given the task.
2. **Critique:** The same LLM analyzes the output and identifies weaknesses.
3. **Refine:** The LLM revises the output based on its own critique.
4. **Repeat:** Steps 2–3 continue until a quality threshold is met or a maximum number of iterations is reached.

This approach has been applied successfully to dialogue response generation, code review, mathematical reasoning, and sentiment reversal. The key insight is that the LLM’s ability to *evaluate* outputs often exceeds its ability to *generate* perfect outputs in a single pass—by separating generation from evaluation, the system can iteratively close the gap.

3.1.2 Reflexion

Reflexion (?) extends self-refinement by introducing a persistent memory of linguistic feedback. The system employs three specialized models:

- **Actor** (M_a): Generates actions or outputs given the task and accumulated memory context.
- **Evaluator** (M_e): Scores the actor’s output, producing a binary reward (success/failure) or scalar score.
- **Self-Reflection** (M_r): Generates verbal feedback explaining what went wrong and suggesting specific improvements.

The core loop operates as follows. For each episode e , the Actor generates action $a_e = M_a(\text{task}, M_{1:e-1})$, the Evaluator scores it $r_e = M_e(a_e, \text{task})$, and if the reward indicates failure, the Self-Reflection module generates feedback $f_e = M_r(a_e, r_e, \text{task})$ which is stored in a growing memory buffer $M_e = M_{e-1} \cup \{f_e\}$.

This “verbal reinforcement learning” achieves 91% pass@1 on HumanEval, surpassing GPT-4’s 80%. The memory buffer acts as a “verbal gradient”—natural language guidance for future attempts that replaces expensive parameter updates.

3.1.3 Self-Debug

Self-Debug (?) uses execution feedback as a first-class self-repair signal. When generated code fails, the system feeds the execution trace (error messages, test outputs, or debugging information) back to the LLM, which then explains the failure and proposes a fix.

The key innovation is treating *execution* as a form of self-evaluation. Unlike Self-Refine (which relies on the LLM’s internal critique) or Reflexion (which relies on linguistic feedback), Self-Debug grounds improvement in concrete runtime behavior.

3.2 Training-Time Self-Improvement

Training-time methods modify the model’s weights, enabling permanent capability improvements that persist across sessions.

3.2.1 SPIN: Self-Play Fine-Tuning

SPIN (?) starts from a supervised fine-tuned (SFT) model and improves it through self-play: the model generates responses, a critic (also the model, from a previous iteration) distinguishes between model-generated and human-generated responses, and the model is fine-tuned to produce responses that are indistinguishable from human ones. This moves from inference-time correction to training-time self-play, enabling cumulative improvement across training iterations.

3.2.2 STaR: Self-Taught Reasoner

STaR (?) generates chain-of-thought rationales for problems, filters those that lead to correct answers, and fine-tunes the model on the filtered set. The key insight is that “reasoning generates reasoning”—the model’s own successful reasoning traces become its training data. Over iterations, the model improves both its reasoning ability and its ability to generate useful rationales.

3.2.3 Constitutional AI

Constitutional AI (?), developed by Anthropic, introduces AI feedback self-alignment. The model generates responses, critiques them based on a set of constitutional principles (e.g., “choose the response that is most harmless”), and revises them accordingly. This creates a self-alignment loop where the AI system improves its own adherence to specified principles without requiring human feedback at each step.

3.2.4 ReST: Reinforced Self-Training

ReST (?) combines self-generation with reinforcement learning. The model generates multiple solutions for each problem, filters by correctness, and fine-tunes on the filtered set using an expectation-maximization approach. This bridges the gap between pure self-improvement (which may amplify biases) and external reward models (which require expensive human annotation).

3.3 Comparative Analysis

Table ?? provides a systematic comparison of self-improvement methods.

Table 2: Comparison of self-improvement methods.

Method	Loop Type(s)	Training?	Feedback Source	Key Result
Self-Refine	Reflection	No	Self-critique	Iterative improvement on dialogue, code, math
Reflexion	Reflection + Evaluator	No	Verbal memory buffer	91% HumanEval pass@1
Self-Debug	Evaluator + Reflection	No	Execution traces	Self-repair from runtime errors
SPIN	Population	Yes	Self-play discriminator	Weak → strong without human data
STaR	Reflection + Evaluator	Yes	Self-generated rationales	Reasoning improves reasoning
Constitutional AI	Reflection	Yes	Constitutional principles	Self-alignment without human feedback
ReST	Search + Evaluator	Yes	Correctness filtering	EM-based self-training

4 Evolutionary Code and Algorithm Discovery

The application of evolutionary principles to code and algorithm discovery represents one of the most exciting frontiers of self-evolving AI. In this paradigm, LLMs serve as intelligent variation operators within evolutionary frameworks, proposing mutations and recombinations of code that are then evaluated by automated scoring functions.

4.1 LLM as Optimizer

4.1.1 OPRO: Optimization by PROMpting

OPRO (?) pioneered the paradigm of using LLMs as general-purpose optimizers. The core insight is that optimization can be formulated as a natural language task: given a history of solutions and their scores, generate a new solution that is likely to score higher.

The system employs a **two-layer prompt architecture**:

- **Outer meta-prompt** (for the optimizer LLM): Contains instruction-score history pairs (sorted by score, top- k retained) plus few-shot examples of the task. The optimizer LLM (temperature=1.0) generates new candidate instructions.
- **Inner task prompt** (for the scorer LLM): Applies the generated instruction to actual task instances. The scorer LLM (temperature=0.0) produces deterministic evaluations.

Key implementation details include: 200 evolution steps, 8 candidates generated per step, MD5 hash-based deduplication, score bucketization (100 discrete bins to prevent sensitivity to minor differences), and wrong-question tracking to dynamically select few-shot examples that highlight the system’s weak points.

OPRO demonstrates cross-task generalization: the same framework successfully optimizes prompts for GSM8K (mathematical reasoning), BBH (27 subtasks), MMLU (57 subtasks), AQuA (algebraic QA), MultiArith (multi-step arithmetic), linear regression parameters, and TSP (traveling salesman problem) routes.

4.2 LLM as Evolutionary Operator

4.2.1 AlphaEvolve

AlphaEvolve (?) represents the current state of the art in LLM-driven evolutionary optimization. It combines several innovations:

1. **Dual-model architecture**: Gemini Flash (fast, broad exploration) generates many candidate mutations, while Gemini Pro (slow, deep insight) proposes targeted improvements to the most promising candidates.
2. **MAP-Elites quality-diversity search**: Instead of maintaining a single best solution, AlphaEvolve maintains an archive of high-performing solutions across behavioral niches. For each behavioral descriptor b , the archive stores the best solution found: $\text{Archive}[b(p)] = p$ if $f(p) > f(\text{Archive}[b(p)])$.
3. **Automated evaluators**: Task-specific scoring functions that can evaluate candidates without human intervention.
4. **Evolution context adaptation**: The prompts provided to the LLM evolve alongside the solutions, incorporating information about successful and failed attempts.

The results are remarkable: AlphaEvolve discovered that 4×4 complex matrix multiplication can be done in **48 multiplications** (the first improvement over Strassen’s algorithm in 56 years). At Google, it recovered 0.7% of worldwide compute through improved data center scheduling (Borg), achieved a 23% speedup in FlashAttention kernel tiling, and a 32% speedup in attention computation.

4.2.2 LLaMEA: Evolving the Optimizer

LLaMEA (?) takes the concept one step further: instead of evolving task solutions, it evolves *meta-heuristic algorithms* themselves. The LLM proposes new optimization algorithms (expressed as code), which are then evaluated on benchmark problems. The best-performing algorithms are retained and serve as parents for the next generation. This is the most direct instantiation of “improving the improver”—the object of evolution is the optimization process itself.

4.3 Program Evolution and Mathematical Discovery

4.3.1 FunSearch

FunSearch (?), published in *Nature*, was the first program discovery system to make scientific discoveries that were deemed significant enough for publication in a top-tier journal. The system combines LLMs with automated evaluators to evolve mathematical programs:

1. Start with a seed program that solves the target problem (possibly poorly).
2. The LLM proposes modifications to the program.
3. An automated evaluator scores each modified program.
4. The best programs are retained and serve as parents for the next iteration.

Key results include new records for the cap set problem and improvements in bin packing algorithms. The fundamental insight is that **evaluator-constrained program evolution can surpass one-shot generation**—the evolutionary process discovers solutions that no single LLM query could produce.

4.4 Open-Source Implementations

Several open-source implementations make evolutionary code optimization accessible:

- **OpenEvolve** (?): (6,358 GitHub stars): Community implementation of AlphaEvolve concepts.
- **SE-Agent**: Multi-trajectory Revision/Recombination/Refinement for code evolution.
- **Science-CodeEvolve**: Specialized for evolving scientific code with domain-specific evaluators.
- **SelfEvolve** (?): Two-step pipeline combining self-generated knowledge (LLM produces its own API documentation) with iterative self-debugging (execution errors feed back for repair). Achieves DS-1000 $49.4 \rightarrow 57.2$ (+15.8% relative) and HumanEval $74.39 \rightarrow 85.98$.

Table ?? compares these systems.

Table 3: Comparison of evolutionary code optimization systems.

System	Year	LLM Role	Search egy	Strat-	Key Result
OPRO	2023	Optimizer	History-guided generation		+8% GSM8K accuracy
FunSearch	2024	Mutation opera- tor	Program evolu- tion	evolu-	New math records (Na- ture)
AlphaEvolve	2025	Mutation + crossover	MAP-Elites QD		48-mult Strassen, 0.7% compute
LLaMEA	2024	Algorithm gener- ator	EA on algorithms		Novel meta-heuristics
OpenEvolve	2024	Mutation opera- tor	Evolutionary search		Community AlphaE- volve
SelfEvolve	2023	Generator + de- bugger	Self-debug loop		+15.8% DS-1000

5 Agent Self-Evolution Systems

Agent self-evolution systems go beyond optimizing individual outputs (prompts, code) to modifying the *agent itself*—its architecture, workflow, memory, and tool usage. This represents the most ambitious form of self-evolution, as the system modifies the very mechanisms that produce its behavior.

5.1 Agent Architecture Self-Evolution

5.1.1 ADAS: Automated Design of Agentic Systems

ADAS (?) automates the search over the design space of agent architectures. Rather than hand-designing agent topologies, ADAS uses a meta-agent that proposes agent architectures, evaluates them on target tasks, and iteratively improves the design. This is analogous to Neural Architecture Search, but the search space is agent architectures rather than neural network topologies.

5.1.2 Gödel Agent

The Gödel Agent (?) is the first self-referential agent framework directly inspired by the Gödel machine. The key insight is that an agent that can read and modify its own source code is “self-referential” in the Gödel sense. The system uses LLM-driven monkey patching to dynamically modify its own logic and behavior at runtime:

1. **Runtime introspection:** The agent reads its own code and current state.
2. **Modification proposal:** The LLM proposes specific code changes (monkey patches).
3. **Safe application:** Changes are applied at runtime via Python monkey patching.
4. **Feedback loop:** Performance feedback guides the next modification.
5. **Rollback:** Failed modifications can be reverted.

The self-referential loop is: $A' = \text{LLM}(A.\text{code}, \text{feedback}, \text{objective})$. Safety mechanisms include rollback capability, validation steps before committing changes, and high-level objective constraints that prevent drift.

5.1.3 Darwin Gödel Machine

The Darwin Gödel Machine (DGM) (?) combines Darwinian evolution with Gödel machine self-reference. It maintains an open-ended archive of agent implementations:

- **Archive:** $A = \{a_1, a_2, \dots, a_n\}$ with fitness scores.
- **Sampling:** $P(a_i) \propto f(a_i) \times \text{diversity_bonus}(a_i, A)$.
- **Modification:** $a_{\text{child}} = \text{LLM}(a_{\text{parent}}.\text{code}, \text{edit_instruction})$.
- **Selection:** Keep if $\text{fitness}(a_{\text{child}}) > \text{fitness}(a_{\text{parent}})$.

The critical innovation is **open-endedness**: unlike standard evolutionary algorithms that converge to a single optimum, DGM maintains a diverse archive, enabling exploration of qualitatively different agent architectures, discovery of unexpected strategies, and cumulative improvement without local optima traps. Results: SWE-bench 20.0% $\rightarrow$ 50.0%, Polyglot 14.2% $\rightarrow$ 30.7%.

5.2 Agent Workflow Self-Evolution

Beyond architecture, the *workflow* of an agent—how it selects and uses tools, sequences actions, and manages its execution flow—can also evolve.

AgentEvolver applies evolutionary optimization to agent tool usage. The system maintains a population of agent configurations with different tool selection strategies and sequencing rules, evaluates them on target tasks, and evolves improved configurations through mutation and crossover of tool-usage patterns.

SE-Agent implements multi-trajectory Revision/Recombination/Refinement. It maintains multiple execution traces for each task, identifies failed trajectories, revises them based on successful patterns, and recombines successful fragments across trajectories.

5.3 Agent Memory and Learning

5.3.1 Agent Symbolic Learning

Agent Symbolic Learning (?) (NeurIPS 2024) treats agents as symbolic networks where the learnable “weights” are prompts, tools, and pipeline composition. The algorithm mimics backpropagation and gradient descent in natural language space:

1. **Forward pass:** Execute the agent on a task, producing a trajectory τ .
2. **Language loss:** $\mathcal{L}_{\text{lang}} = \text{LLM}(\mathcal{P}_{\text{loss}}(\tau))$.
3. **Language gradient backpropagation:** For each layer n (from output to input), compute $\nabla_{\text{lang}}^n = \text{LLM}(\mathcal{P}_{\text{gradient}}(\nabla_{\text{lang}}^{n+1}, \text{node_info}_n, \mathcal{L}_{\text{lang}}))$.
4. **Weight update:** PromptOptimizer, ToolOptimizer, and PipelineOptimizer update their respective “weights” using the language gradients.

The “language gradient” is a natural language description of how to improve each component, propagated backward through the agent’s computational graph—analogous to backpropagation but operating entirely in text space. Results: HotPotQA F1 39.1 (vs. 36.2 baseline), MATH 60.7% (vs. 56.0%), HumanEval 73.2% (vs. 68.3%).

5.3.2 Absolute Zero

Absolute Zero (?) (NeurIPS 2025 Award) eliminates the dependency on human-curated training data entirely. A single model simultaneously proposes tasks and solves them using only self-generated data:

1. **Task Proposal:** The model generates new reasoning tasks (code, math, logic).
2. **Task Solving:** The model attempts to solve its own proposed tasks.
3. **Code Executor Verification:** Automated evaluation via code execution provides the reward signal.
4. **RL Training:** Both the proposer and solver are updated via reinforcement learning (PPO/GRPO).

The model is **both** the curriculum designer and the student. It must propose tasks that are solvable (not trivially impossible), challenging (not trivially easy), and diverse (covering different reasoning skills). This represents the frontier of self-evolution: a system that creates its own learning curriculum and improves through self-play.

5.4 Multi-Agent Collaborative Evolution

AutoML-Agent (?) demonstrates multi-agent division of labor for self-evolution. Specialized agents handle data retrieval, preprocessing, modeling, evaluation, and deployment. The system combines retrieval-augmented planning with multi-stage validation, making the evolution process itself a collaborative multi-agent activity.

SelfEvolve (?) provides a complete self-evolution pipeline that combines self-generated knowledge (Stage 1: LLM produces its own API documentation) with iterative self-debugging (Stage 2: execution errors feed back for repair). This two-stage approach addresses a key challenge in self-evolution: the system must first understand its own knowledge boundaries before it can effectively improve.

6 Agent Frameworks and Infrastructure

Agent frameworks provide the orchestration layer that connects LLMs with tools, memory, and evaluation. While not all frameworks are designed for self-evolution, understanding their capabilities and gaps is essential for building self-evolving systems.

6.1 Framework Overview and Comparison

The current agent framework landscape includes several major systems, each with different design priorities:

- **AutoGPT (?)**: Goal-driven autonomous agent platform that popularized the concept of LLM agents with reusable protocols. Includes agent evaluation infrastructure but lacks persistent evolution mechanisms.
- **MetaGPT (?)**: Encodes human software company workflows as SOP-driven multi-agent collaboration. Role specialization with structured processes stabilizes output. Key insight: “evolution” also needs organization, roles, and process memory.
- **AutoGen (?)**: Supports conversational, customizable agents with human-in-the-loop, tool usage, and multi-agent workflow patterns. Provides flexible agent conversation infrastructure.
- **DSPy (?)**: Declarative LLM programming paradigm that compiles declarative language model calls into self-improving pipelines. Unique in treating prompt optimization as a compilation problem.
- **CAMEL (?)**: Communicative agents for mind exploration through role-playing. Provides multi-agent conversation primitives.

Table ?? provides a systematic comparison.

Table 4: Comparison of agent frameworks.

Framework	Orchestration	Memory	Tools	Evaluation	Evolution
AutoGPT	Goal-driven	Short-term	Yes	Basic	Limited
MetaGPT	SOP-driven	Process	Yes	Role-based	None
AutoGen	Conversation	Configurable	Yes	External	None
DSPy	Declarative	Pipeline state	Yes	Built-in	Prompt opt.
CAMEL	Role-play	Conversation	Limited	External	None
CrewAI	Crews + flows	Configurable	Yes	External	None

6.2 Evolution Capability Gaps

Current agent frameworks share several critical gaps that prevent self-evolution:

1. **Persistent evaluation layer**: Most frameworks evaluate per-run, without accumulating evaluation history across runs. There is no mechanism to track whether a change improved or degraded performance over time.
2. **Lineage and regression protection**: No built-in mechanism for tracking which changes led to which improvements, or for preventing regressions. A system that optimizes for the current task may lose capabilities on previous tasks.
3. **Cross-run memory**: Agents do not accumulate experience across multiple deployments or tasks. Each run starts from scratch, wasting the information gained from previous executions.

6.3 Self-Evolve Positioning: Evolution Layer Above Frameworks

Self-Evolve is positioned not as a replacement for existing agent frameworks (AutoGPT, MetaGPT, AutoGen, CrewAI) but as an **evolution layer that sits above them**. The five loops (specification-to-execution, search, evaluator, reflection, population) can be applied to any agent framework, adding the persistent evaluation, memory, lineage, and improvement mechanisms that current frameworks lack.

This architecture mirrors the relationship between operating systems and applications: frameworks provide the execution environment, while Self-Evolve provides the evolution infrastructure that monitors, evaluates, and improves the agents running within that environment.

7 AutoML and NAS Meets LLM

The intersection of automated machine learning (AutoML), neural architecture search (NAS), and large language models represents a particularly productive area of self-evolution research. These communities have independently developed mechanisms that align closely with the five evolution loops.

7.1 AutoML + LLM

AutoML-GPT (?) uses GPT as a natural language bridge, converting task cards, model cards, and data cards into data processing, model architecture, hyperparameter optimization, and training log prediction. This extends AutoML from fixed toolboxes to prompt-driven planning and execution pipelines.

AutoML-Agent (?) decomposes the full AutoML pipeline into specialized agents for data retrieval, preprocessing, modeling, evaluation, and deployment. It combines retrieval-augmented planning with multi-stage validation, making AutoML a collaborative multi-agent activity.

The key insight from this direction is that AutoML + LLM is the most **production-ready** form of self-evolution: task input, pipeline synthesis, evaluation, and deployment are all end-to-end systematized. The main gap is **continuous evolution**: most systems automate single projects, while Self-Evolve emphasizes cross-run benchmark memory and population-level improvement.

7.2 NAS + LLM

EvoPrompting (?) treats LLMs as evolutionary operators for architecture code, evaluating candidate graph neural networks on algorithmic reasoning tasks. LLMatic (?) starts from simple networks and uses LLM-guided search with quality-diversity optimization to explore the architecture space. NADER (?) represents neural architecture design as LLM-based multi-agent collaboration, connecting NAS with agent specialization.

NAS + LLM provides the strongest technical metaphor for self-evolution: **architecture/code is the evolvable genome**. Quality-diversity and multi-objective search should enter the Self-Evolve vocabulary: lineage, fitness, novelty, robustness, and deployment cost.

7.3 Unified Perspective with Self-Evolution

AutoML and NAS represent two faces of the same evolution loop: AutoML evolves pipelines (search + specification loops), while NAS evolves architectures (search + evaluator loops). Both can benefit from the reflection and population loops that Self-Evolve provides. The unification of these communities under a single taxonomy reveals shared principles and opportunities for cross-pollination.

8 Research Landscape and Community

The field of AI self-evolution is shaped by a tightly interconnected network of research groups spanning industry labs, academic institutions, and open-source communities.

8.1 Core Research Groups

Google Research and DeepMind have been the most prolific contributors: OPRO (?), FunSearch (?) (published in *Nature*), and AlphaEvolve (?) all originate from this ecosystem. The DeepMind lineage is notable for connecting evolutionary computation with LLM capabilities.

Stanford University produced foundational work on self-improvement through reasoning: STaR (?) from the Noah Goodman lineage, and contributions to the self-refinement paradigm.

Princeton NLP, led by Karthik Narasimhan, developed Reflexion (?) and the ReAct/Tree of Thoughts frameworks that provide foundational primitives for agent reasoning.

UBC, through Jeff Clune’s group, pioneered open-ended evolutionary approaches with the Darwin Gödel Machine (?) and ADAS (?).

Tsinghua University contributed Absolute Zero (?) (NeurIPS 2025 Award), demonstrating zero-data self-improvement.

Anthropic developed Constitutional AI (?), focusing on the safety dimensions of self-improvement.

8.2 Cross-Community Convergence

A striking pattern is that different research communities independently discover similar mechanisms: AutoML’s “task card $\rightarrow$ pipeline synthesis” mirrors the agent community’s “goal $\rightarrow$ plan $\rightarrow$ execute”; NAS’s “architecture mutation $\rightarrow$ fitness evaluation” mirrors evolutionary computation’s “LLM-as-variation-operator”; and LLM self-improvement’s “generate $\rightarrow$ critique $\rightarrow$ refine” mirrors the agent community’s “act $\rightarrow$ reflect $\rightarrow$ retry.” This convergence suggests that the five evolution loops are fundamental attractors in the design space of self-improving systems.

8.3 Open-Source Ecosystem

Table ?? summarizes the key open-source projects.

Table 5: Open-source projects in AI self-evolution.

Project	Stars	Evolution Mode	License
OpenEvolve	6,358	Search + Evaluator	Apache 2.0
aiwaves/Agents	5,928	Search + Eval + Agent	Apache 2.0
Reflexion	3,158	Search + Reflection + Eval	MIT
AgentEvolver	1,441	Search + Eval + Agent + Train	Apache 2.0
Self-Refine	805	Reflection + Refinement	MIT
SE-Agent	274	Search + Eval + Agent	MIT
Science-CodeEvolve	97	Search + Evaluator	MIT
SCOPE	77	Search Loop	MIT
LLM-Self-Judge	43	Search + Eval + Agent	Apache 2.0
DARWIN	41	Search + Reflection	MIT

9 Open Problems and Future Directions

Despite rapid progress, significant challenges remain before self-evolving AI systems can be deployed reliably in production.

9.1 The Evaluation Bottleneck

Benchmark improvements do not necessarily translate to real-world reliability. Current systems optimize against specific benchmarks (HumanEval, SWE-bench, GSM8K), creating implicit overfitting risks. As systems saturate standard benchmarks, the marginal information gain from each evaluation decreases, and the gap between benchmark and real-world distributions may widen. **Direction:** Multi-dimensional evaluation frameworks combining benchmarks, deployment metrics, safety checks, and human evaluation.

9.2 Memory Stability and Drift

Self-evolving systems accumulate experience over time, but this memory introduces stability challenges: catastrophic forgetting (losing previously mastered capabilities), memory drift (unintended behavioral shifts), and unbounded growth (Reflexion’s memory buffer, Agent Symbolic Learning’s gradient history). **Direction:** Principled memory management with forgetting curves, experience replay, and selective retention.

9.3 Safety and Alignment

Self-modifying systems raise unique safety concerns: objective drift (inadvertently changing the objective function), reward hacking (maximizing evaluation scores without genuine improvement), and unbounded modification (escaping human oversight). The Gödel Agent’s rollback mechanism and Constitutional AI’s principle-based constraints represent early progress. **Direction:** Safety-constrained self-evolution with invariant properties and controlled adaptation boundaries.

9.4 Composability of Evolution Loops

The five loops can be composed in various configurations, but their interactions are poorly understood. Does a reflection loop on top of a population loop produce compounding benefits or interference? How should resources be allocated across multiple active loops? **Direction:** Formal frameworks for loop composition with theoretical guarantees.

9.5 From Research to Production

Most systems operate in controlled settings. Production deployment introduces latency constraints, LLM API cost accumulation, observability requirements, and version control challenges. **Direction:** Production-grade infrastructure with cost controls, regression gates, version tracking, and deployment safeguards.

9.6 The Self-Evolve Roadmap

1. **Phase 1 — Instrumentation** (current): Instrument existing frameworks with evaluation layers, lineage tracking, and cross-run memory.
2. **Phase 2 — Single-loop evolution:** Deploy individual loops (reflection, population-based optimization) with regression gates.
3. **Phase 3 — Multi-loop composition:** Combine multiple loops with principled resource allocation and safety constraints.
4. **Phase 4 — Open-ended evolution:** Systems that discover new evolution strategies, expanding their own design space.

10 Conclusion

This survey has presented a unified view of AI self-evolution through the lens of five fundamental loops: specification-to-execution, search, evaluator, reflection, and population. Our analysis of 19+ open-source projects and 17+ foundational papers across five research communities reveals a striking convergence: AutoML researchers, NAS designers, LLM self-improvement engineers, evolutionary computation scientists, and agent framework developers are all independently rediscovering the same fundamental mechanisms for self-improvement.

The key insight is that self-evolving AI is not a single technique or a property of any one research community. Rather, it is an *emergent capability* that arises when any system incorporates the four primitives of generation, evaluation, memory, and improvement into an iterative cycle. The five loops represent different configurations of these primitives, each suited to different substrates and timescales.

The frontier is moving from fixed-architecture self-improvement toward open-ended self-modification. Systems like the Darwin Gödel Machine maintain open-ended archives of agent implementations; the Gödel

Agent uses self-referential code modification; and Absolute Zero demonstrates that self-improvement can proceed with zero external data. These developments suggest that the next wave of AI systems will not just improve within fixed design spaces but will expand their own design spaces over time.

However, significant challenges remain: the evaluation bottleneck limits our ability to verify genuine improvement; memory stability threatens long-term reliability; safety alignment in self-modifying systems is an open problem; and the composability of multiple evolution loops is poorly understood. Addressing these challenges will require new infrastructure for production self-evolution—evaluation layers above agent frameworks, lineage tracking across runs, regression gates for self-modifying code, and cost controls for LLM-driven evolution cycles.

We position Self-Evolve as the missing layer: not a replacement for existing agent frameworks but an evolution infrastructure that sits above them, providing the persistent evaluation, memory, lineage, and improvement mechanisms that current frameworks lack. The five loops provide a design vocabulary for building this layer, and the projects analyzed in this survey provide concrete implementation patterns for each loop.

A Benchmark Data Summary

Table 6: Key benchmark results across surveyed systems.

System	Benchmark	Metric	Score
Reflexion	HumanEval	pass@1	91%
GPT-4 (baseline)	HumanEval	pass@1	80%
SelfEvolve	HumanEval	pass@1	85.98%
SelfEvolve	DS-1000	pass@1	57.2
DGM	SWE-bench	Verified	20% → 50%
DGM	Polyglot	Score	14.2% → 30.7%
Agent Sym. Learning	HotPotQA	F1	39.1
Agent Sym. Learning	MATH	Accuracy	60.7%
Agent Sym. Learning	HumanEval	pass@1	73.2%
AlphaEvolve	4 × 4 matrix mult.	Multiplications	48
AlphaEvolve	FlashAttention	Speedup	23%
AlphaEvolve	Google compute	Recovery	0.7% worldwide
OPRO	GSM8K	Accuracy	+8% over baseline
FunSearch	Cap set problem	Result	New record
FunSearch	Bin packing	Result	Beyond SOTA

B Core Papers Analyzed

Table 7: Core papers analyzed with self-evolution relevance.

#	Paper	Key Contribution	Relevance
1	Agent Symbolic Learning	Symbolic backprop for agents	High
2	Darwin Gödel Machine	Open-ended agent evolution	Very High
3	Gödel Agent	Self-referential modification	High
4	ADAS	Automated agent design search	High
5	Reflexion	Verbal reinforcement learning	Very High
6	Self-Refine	Iterative self-feedback	High
7	Absolute Zero	Zero-data self-play	Very High
8	AlphaEvolve	Gemini + MAP-Elites	Very High
9	RISE	Recursive self-improvement	High
10	RAGen	RAG-enhanced generation	Medium-High
11	SelfEvolve	Code evolution framework	Very High
12	ReVeal	Verification-guided evolution	High